Error Boundaries are React components that catch and handle runtime errors within their child component tree. They provide a mechanism for rendering a fallback UI when an error occurs, preventing the entire application from crashing and significantly improving the user experience.

There are three key features of Error Boundaries:

- Catch errors during rendering, lifecycle methods, and constructors of child components.
- Display a fallback UI when an error is detected.
- Log error details for debugging and observability.

Listing A.5 demonstrates a basic Error Boundary component, which uses two lifecycle methods:

- `getDerivedStateFromError`—A static method that updates the state to trigger the rendering of a fallback UI.
- `componentDidCatch`—An instance method that logs the error details.

Listing A.5 Error Boundary component

```
class ErrorBoundary extends Component<ErrorBoundaryProps, ErrorBo
  constructor(props: ErrorBoundaryProps) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error: Error): ErrorBoundarySta
    return { hasError: true, error };
  }
  componentDidCatch(error: Error, errorInfo: ErrorInfo): void { #
    console.error('Error caught by ErrorBoundary:', error);
    console.error('Error details:', errorInfo.componentStack);
  }
  render(): ReactNode {
    if (this.state.hasError) { #C
      return (
        this.props.fallback || (
          <div>
            <h1>Something went wrong.</h1>
            {this.state.error && (
              <details style={{ whiteSpace: 'pre-wrap' }}>
                {this.state.error.toString()}
```